

ATTENZIONE! QUESTO MATERIALE DIDATTICO È PER USO PERSONALE DELLO STUDENTE
ED È COPERTO DA COPYRIGHT. NE È VIETATA LA RIPRODUZIONE O IL RIUTILIZZO ANCHE PARZIALE,
AI SENSI E PER GLI EFFETTI DELLA LEGGE SUL DIRITTO D'AUTORE.

Statecharts

Ingegneria del Software Avanzata

Università di Ferrara

CdLM in Ingegneria Informatica e dell'Automazione

A.A. 2021/2022

Marco Alberti (lbrmrc@unife.it)

Introduzione

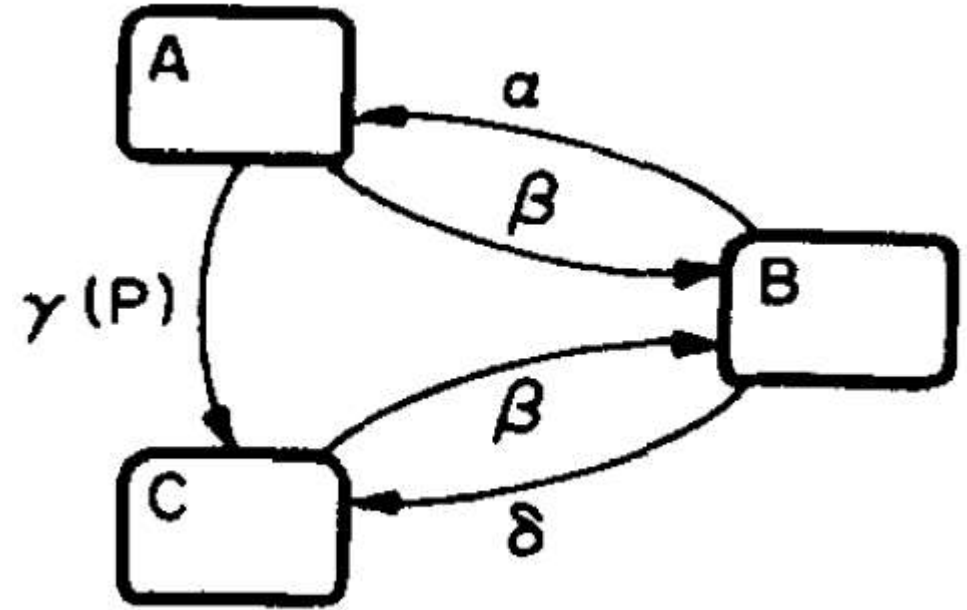
- David Harel, «Statecharts: a visual formalism for complex systems», Science of Computer Programming 8(3), 1987
- Motivazione: arricchire gli automi a stati finiti con elementi utili a rappresentare sistemi complessi.
- Adottati da UML (state diagrams)
- A rigore, linguaggio semiformale: semantica non completamente specificata (varie proposte)

Motivazione

- Le FSM soffrono di esplosione dello spazio degli stati per sistemi gerarchici.
- Sono inoltre poco adatte ad esprimere requisiti come i seguenti:
 - «in volo, quando si tira la leva gialla il sedile viene espulso»
 - «la marcia innestata è indipendente dallo stato dell'impianto frenante»
 - «in seguito alla pressione del pulsante di selezione, entrare in modalità selezione»
 - «la modalità visualizzazione è composta dalle modalità di visualizzazione dell'orario, visualizzazione della data e visualizzazione del cronometro»
- Statecharts = FSM + gerarchia + ortogonalità + broadcast

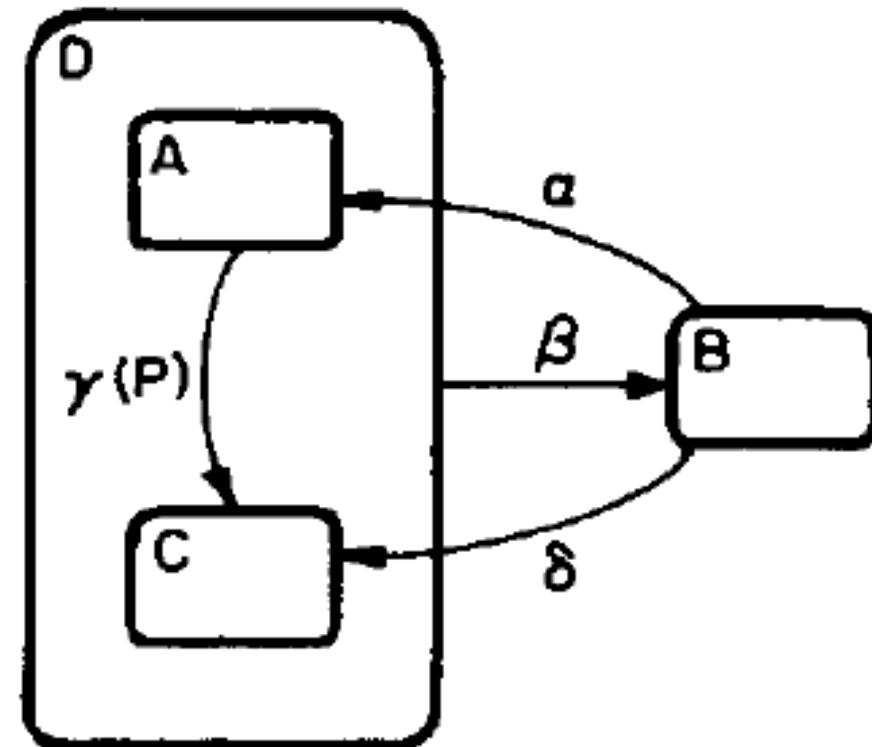
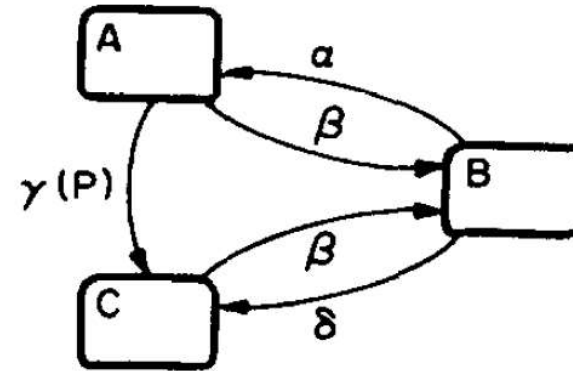
Rappresentazione grafica

- FSM rappresentate come grafo diretto
- Statechart come higraph
- Stati: rettangolo arrotondato
- Eventi (con condizioni): arco
- Utilizzano l'area: a differenza delle FSM, gli stati hanno estensione spaziale e la loro posizione è significativa



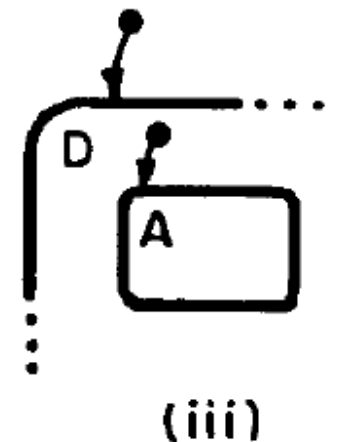
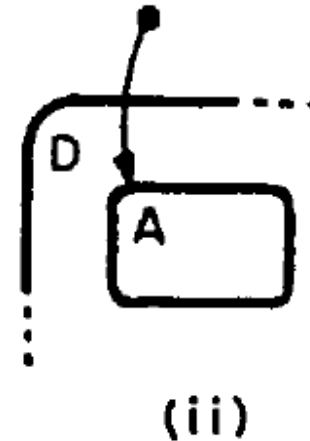
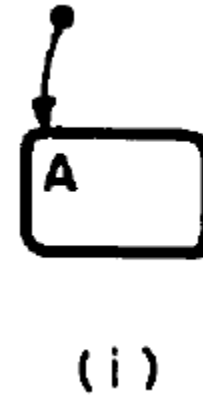
Gerarchia

- Uno stato che ne contiene altri è un loro *superstato*.
- Gli stati contenuti direttamente nello stesso superstato sono un gruppo.
- Il sistema si trova in un superstato se e solo se si trova in esattamente uno degli stati del gruppo.
- Un arco uscente dal superstato è equivalente all'insieme di archi con lo stesso stato di arrivo uscenti da tutti gli stati del gruppo



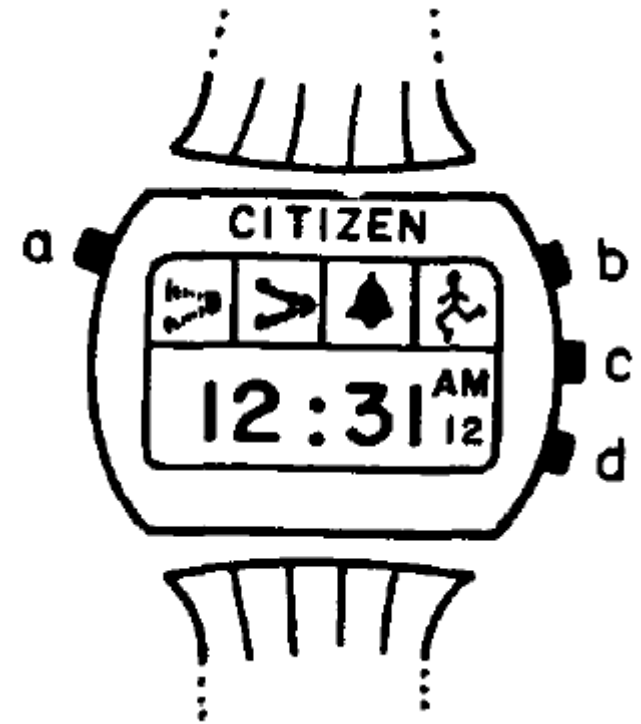
Default state

- Uno stato in un gruppo di stati è di default se è puntato da un arco senza stato di partenza (come A nella figura (i)).
- Salvo specifica contraria, il sistema entra negli stati di default
 - a inizio esecuzione
 - quando entra in un superstato: le figure (ii) e (iii) sono equivalenti.



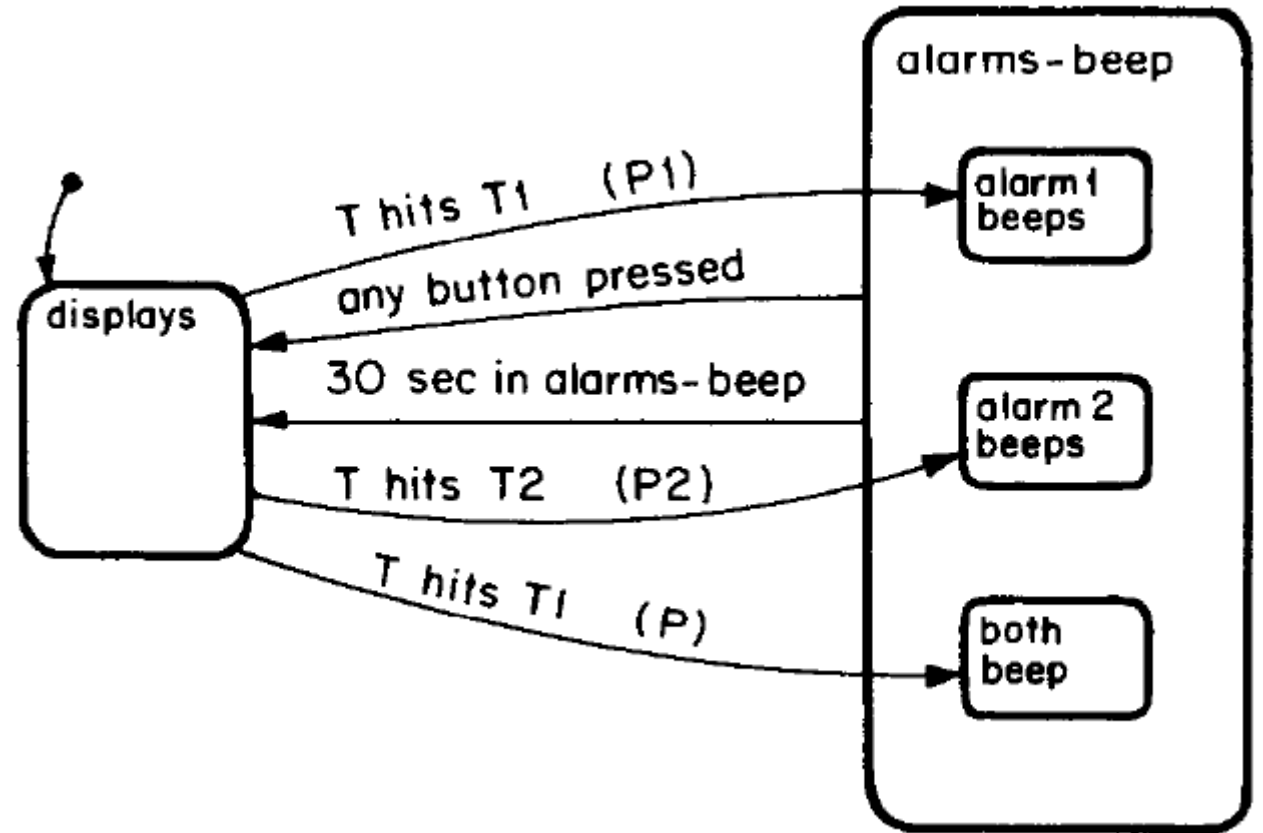
Esempio

- Citizen Multi Quartz Alarm III (1979)
- Utilizzato in Harel (1987) come running example
- Statechart completo: figura 31 dell'articolo



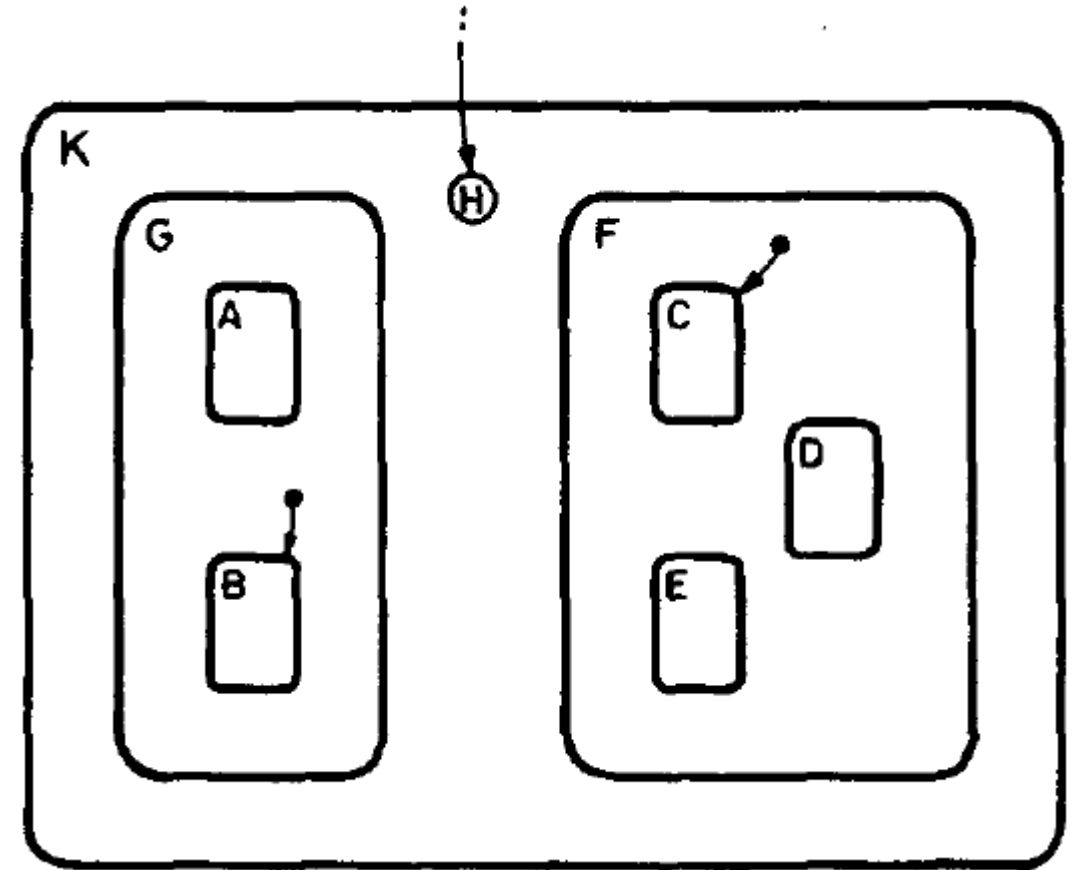
Esempio gerarchia

- T1, T2: ora primo e secondo allarme
- T: ora attuale
- P1: allarme 1 attivato e, inoltre, allarme 2 disattivato o $T1 \neq T2$
- P2 idem
- P: allarmi 1 e 2 attivati e $T1 = T2$



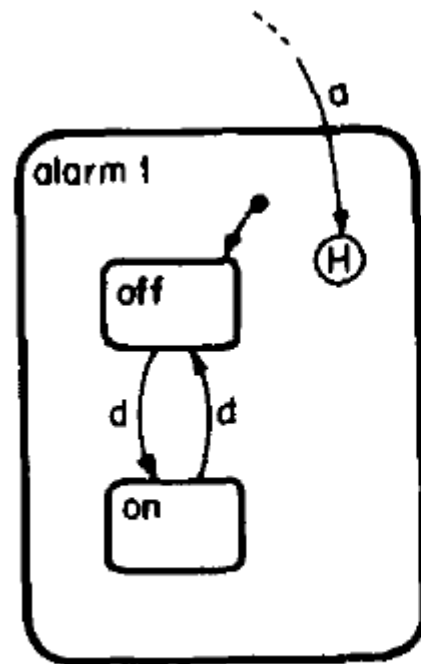
History

- Uno stato senza estensione marcato con H (history) significa che l'ingresso nel gruppo di stati a cui appartiene avviene nell'ultimo stato visitato.
- Nella figura, se il sistema entra in K, entrerà anche in
 - G (e quindi nel default B) se prima era in A o B
 - F (e quindi nel default C) se prima era in C, D, o E

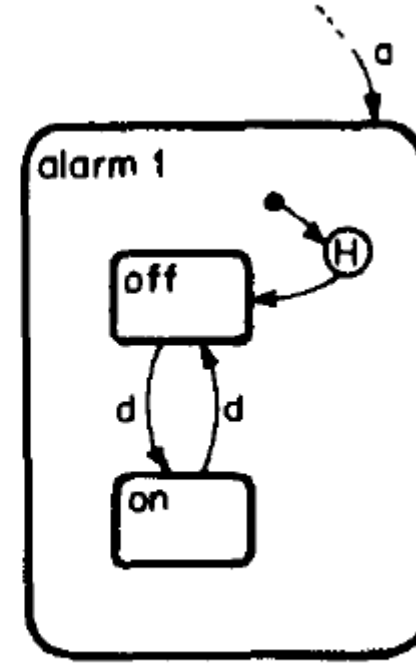


Esempio: attivazione/disattivazione allarme

- Specifiche equivalenti



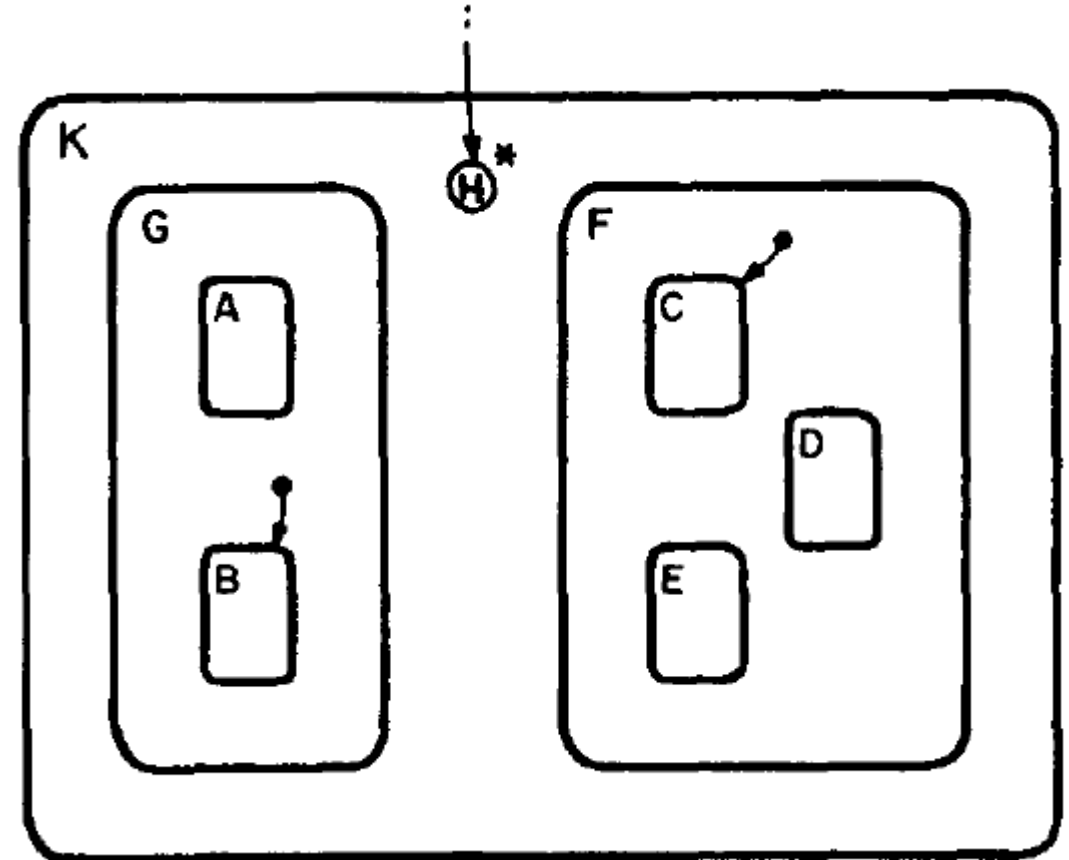
(a)



(b)

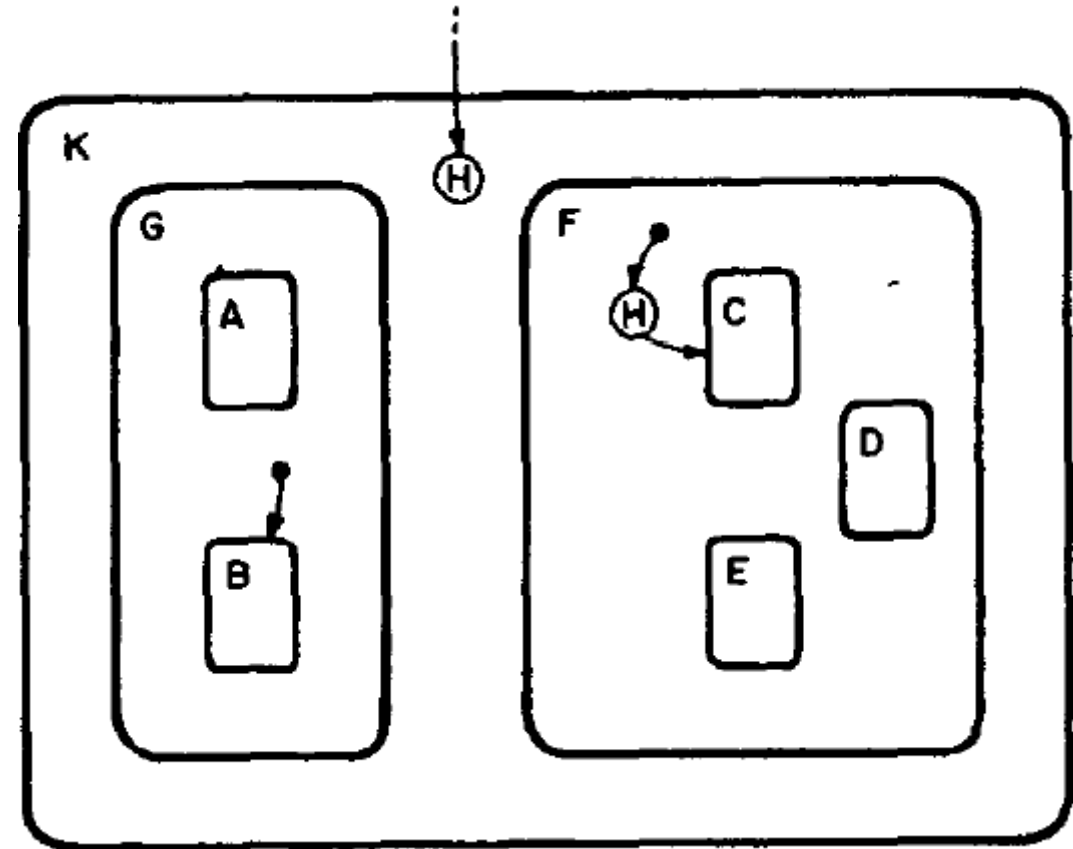
History

- Normalmente l'accesso by history si applica solo al livello in cui si trova lo stato H.
- Con l'asterisco si indica che l'accesso by history si applica fino agli stati più interni (annullando gli stati di default).
- Nella figura, all'ingresso in K tornerà attivo lo stato più interno (A, B, C, D, E o F) che era attivo all'uscita da K.



History

- Osservazione: è possibile specificare l'ingresso nei sottostati by history anche solo per alcuni degli stati di un gruppo.
- In questo caso, all'ingresso in K, il sistema entra in
 - B se all'uscita da K lo stato era A o B
 - se all'uscita da K lo stato era C, D o E, lo stesso stato

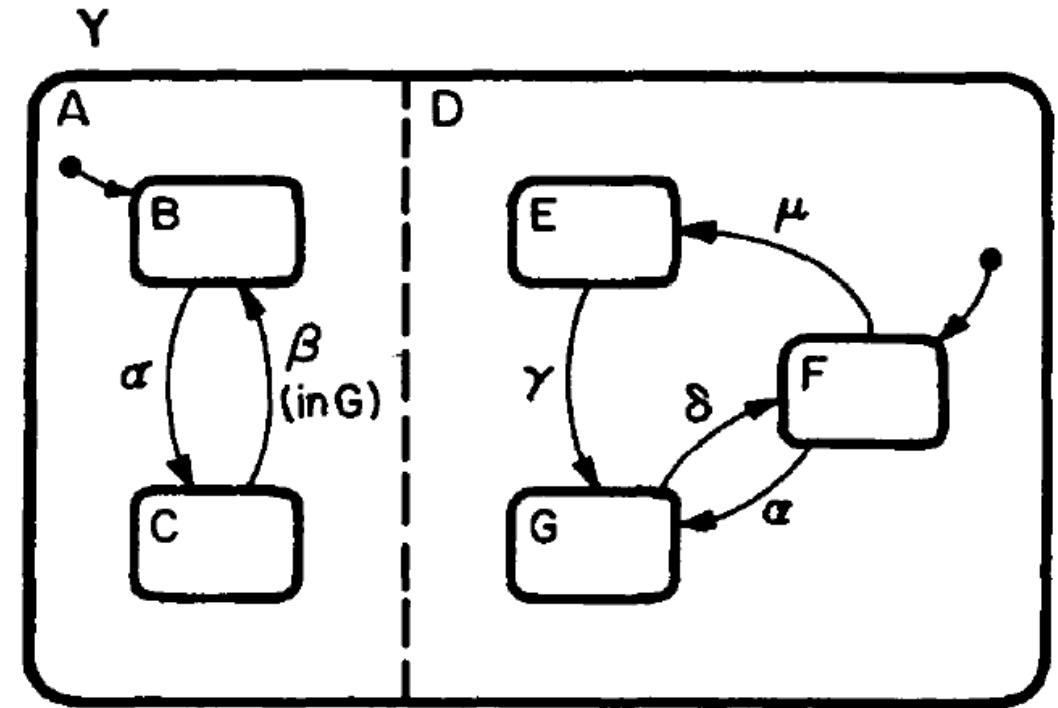


Decomposizione in OR e AND

- Finora abbiamo visto gli stati decomposti in OR (esclusivo): in ogni gruppo di stati, uno e soltanto uno è attivo in ogni istante.
- Gli stati si possono rappresentare come un albero: lo stato A contiene lo stato B se e solo se b è figlio di A.
- Solo una foglia, e tutti i nodi del percorso da quella foglia alla radice, rappresentano gli stati attivi in un istante.
- Vedremo ora che gli stati si possono decomporre anche in AND, cioè possono essere scomposti in gruppi in ognuno dei quali uno stato è attivo.

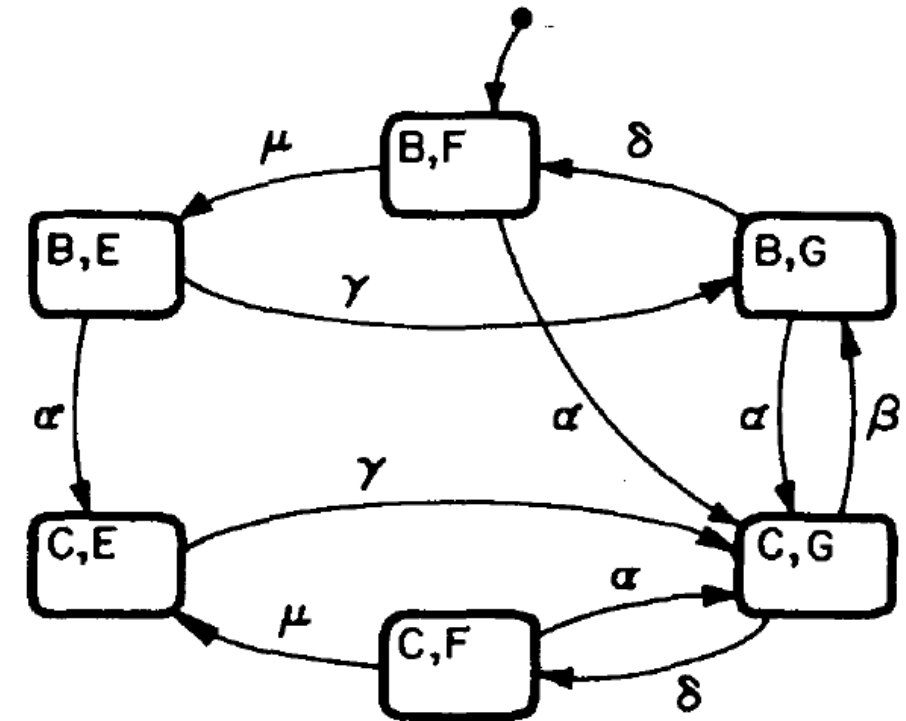
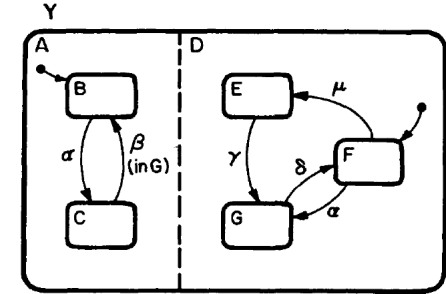
Ortogonalità

- Graficamente: due o più gruppi sono in AND (o ortogonali) se sono separati da una linea tratteggiata che divide loro il superstato.
- In ogni istante uno stato di ogni gruppo è attivo.
- Nella figura, le possibili combinazioni sono BE, BF, BG, CE, CF, CG; i superstati A e D sono entrambi attivi e compongono Y.



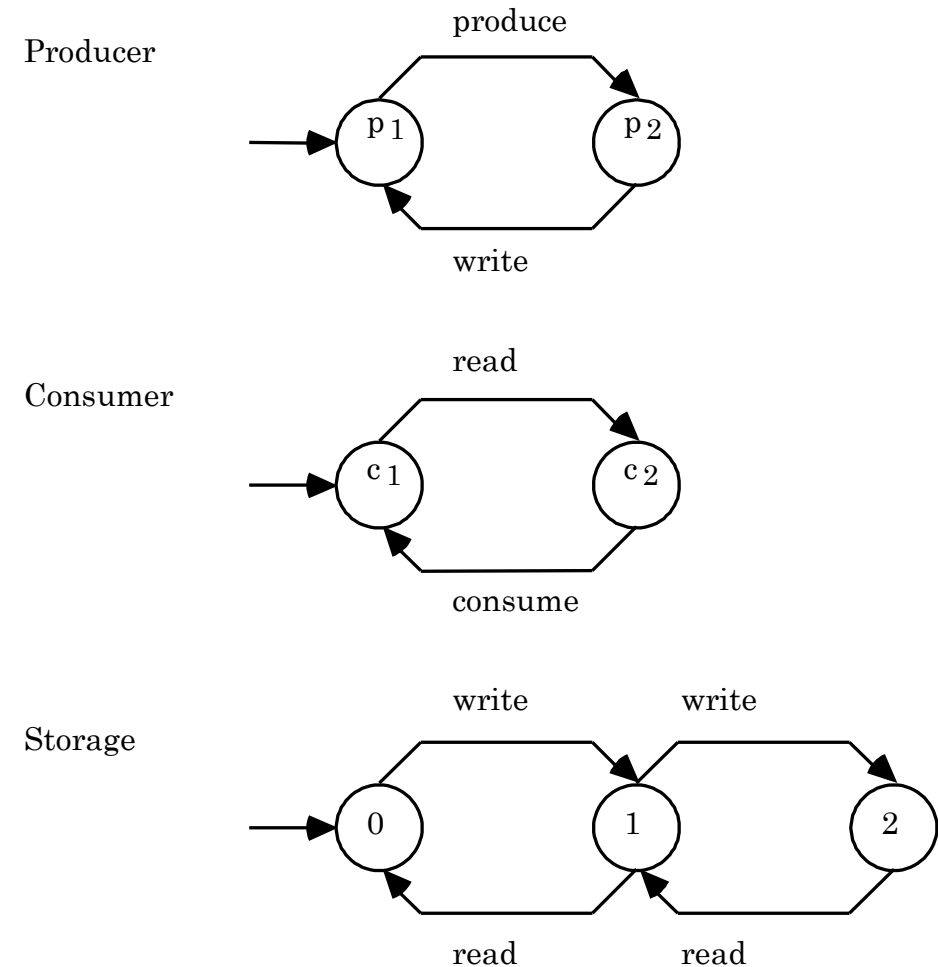
Migliore scalabilità rispetto a FSM

- L'insieme dei possibili stati attivi è il prodotto cartesiano dei gruppi di stati in AND.
- In figura è mostrato l'automa a stati finiti equivalente. Notare:
 - più stati (2x3 anziché 2+3)
 - più archi (ad esempio γ è ripetuto)
 - alla condizione (in G) dell'arco da C a B nello statechart corrisponde l'assenza degli archi β CE-BE e CF-BF

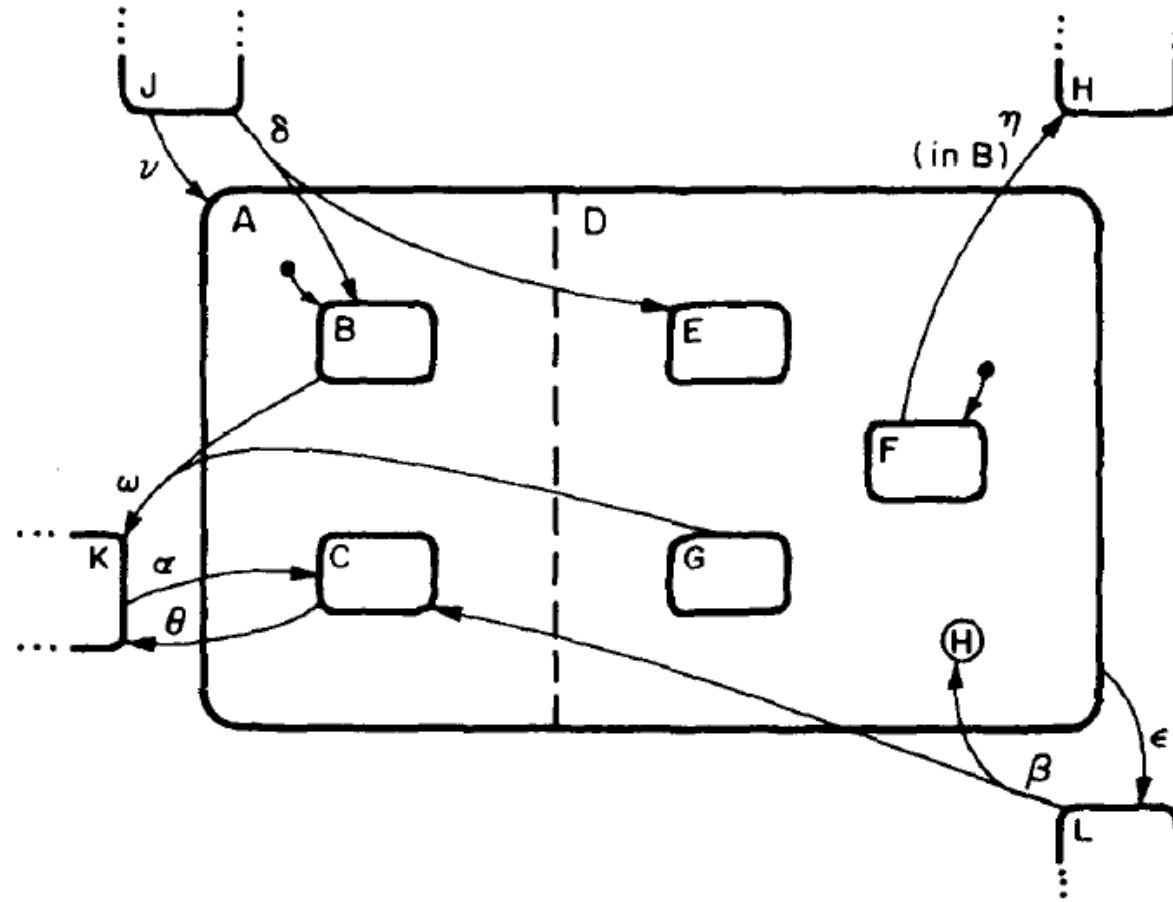


Esempio: Produttore-Consumatore

- In figura sono mostrate le tre FSM per il sistema produttore-consumatore.
- Possono anche essere interpretate come statechart per i tre sottosistemi.
- Lo statechart complessivo è semplicemente l'AND degli statechart dei tre componenti.

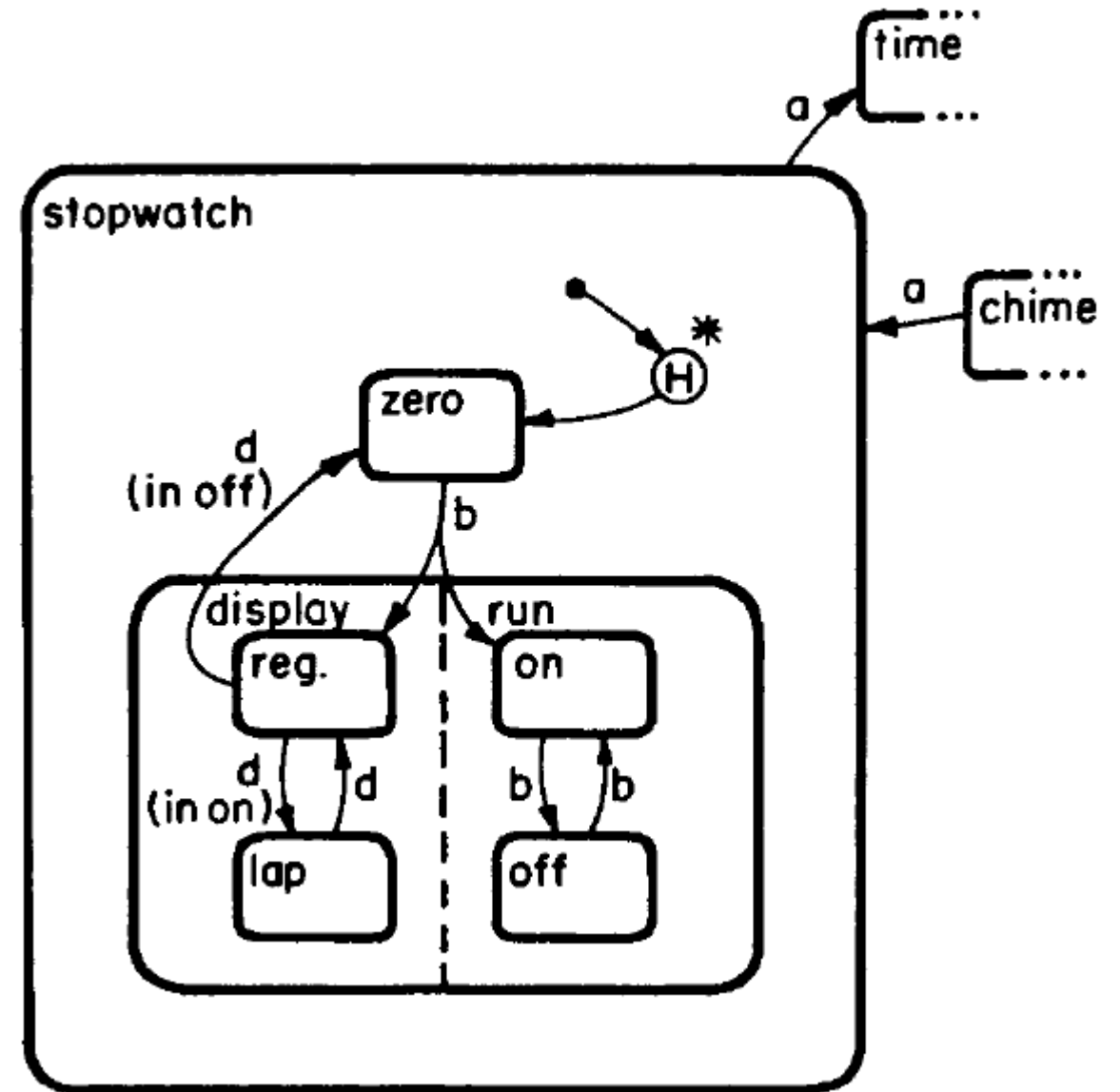


Ingresso/uscita e stati ortogonalità

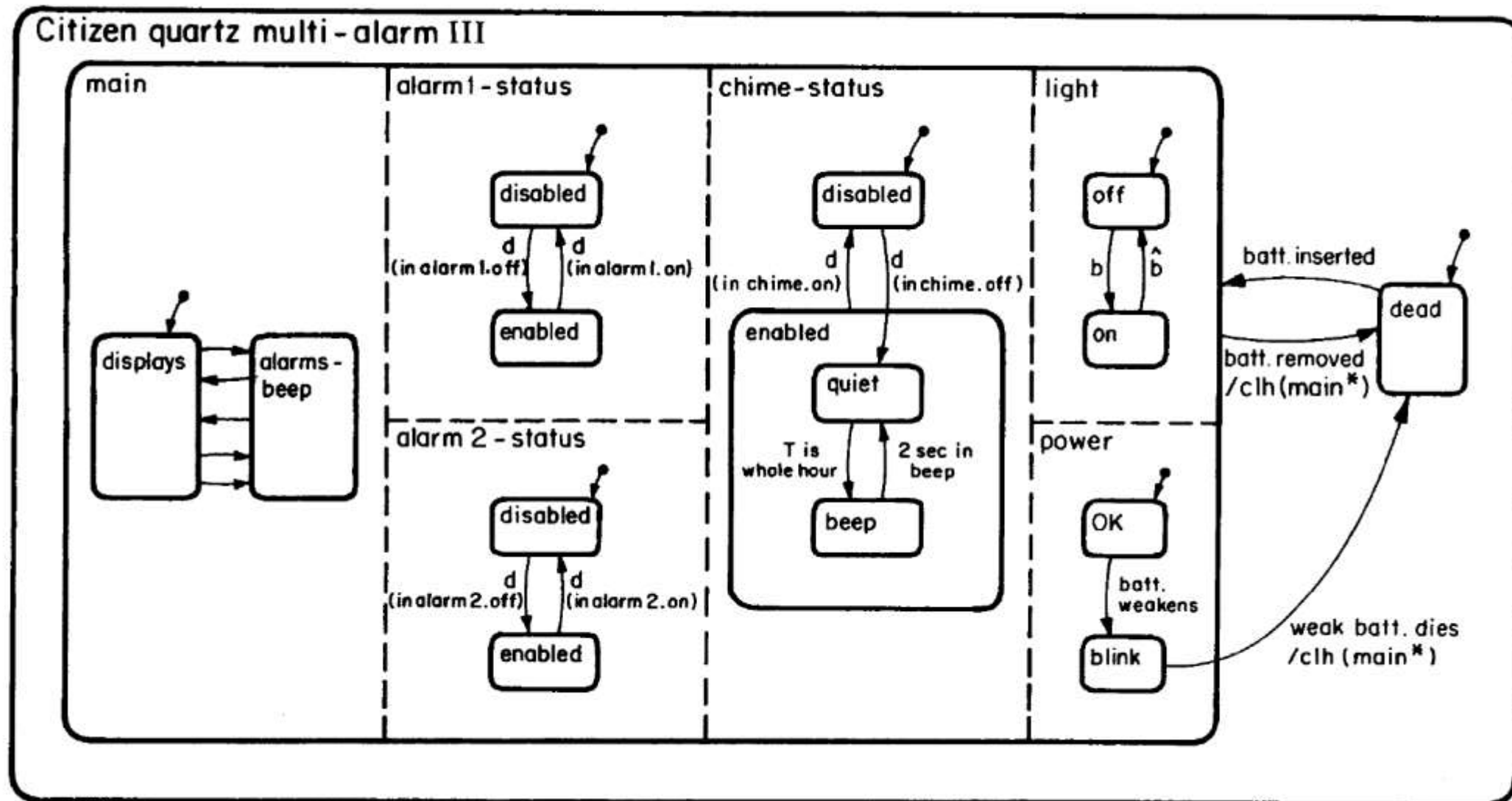


Esempio: cronometro

- Il cronometro ha due stati in OR: zero o in funzione
- Lo stato in funzione è composto da due stati in AND: display (normale o tempo parziale) e run (on o off).



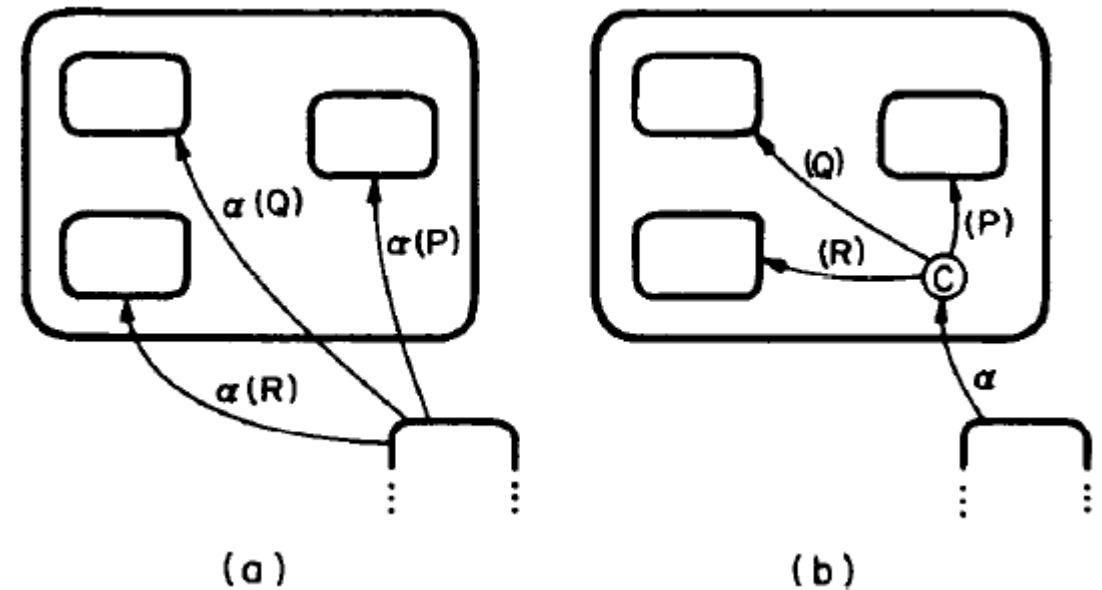
Esempio: stato complessivo



Ingresso per condizione

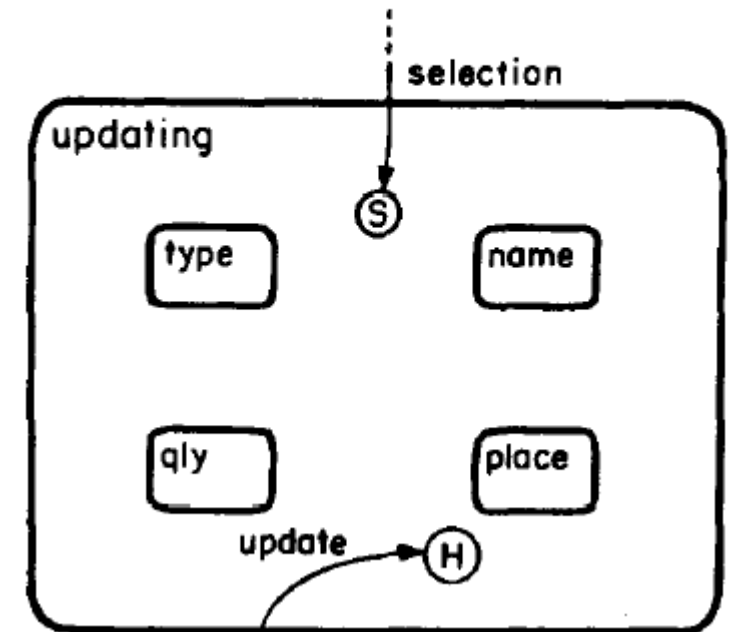
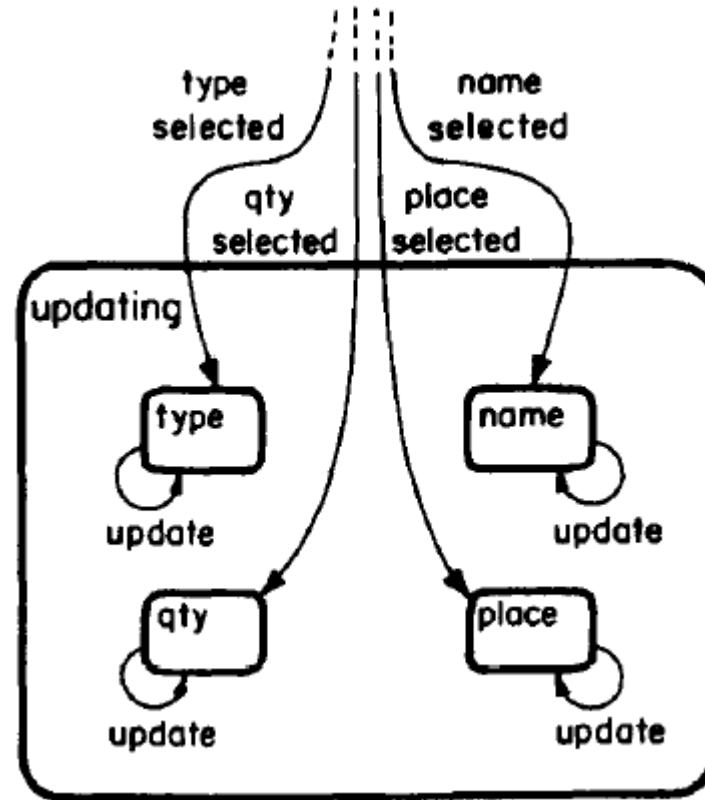
- Stato senza estensione contrassegnato con C
- La combinazione
 - arco con segnale entrante in stato condizione
 - n archi con condizioni uscenti da stato condizione

corrisponde ad n archi con lo stesso segnale e le stesse n condizioni, con gli stessi stati di arrivo



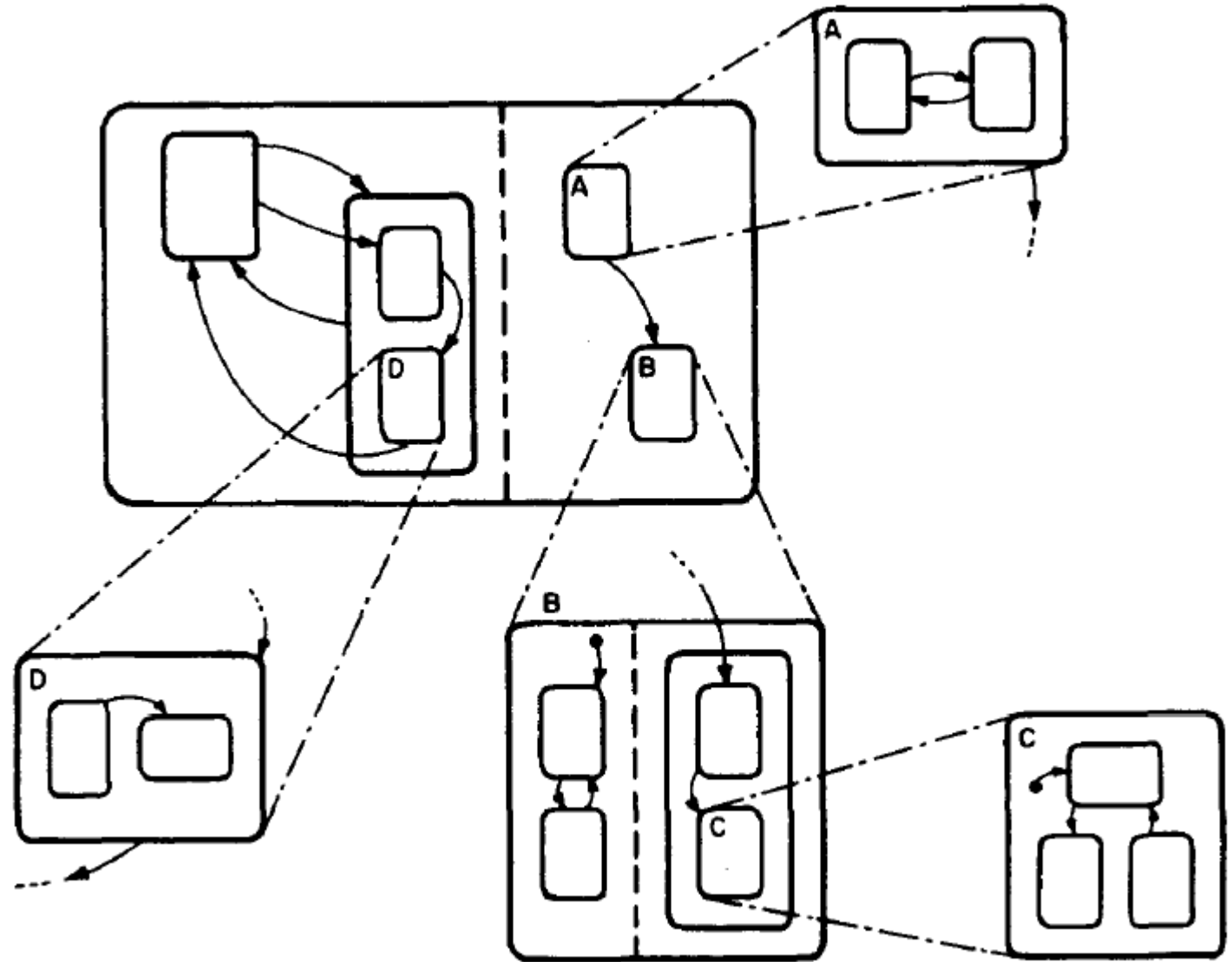
Selection

Applicabile quando
c'è corrispondenza
biunivoca fra lo stato
di arrivo e l'evento, o
fra lo stato di arrivo e
un parametro
dell'evento.



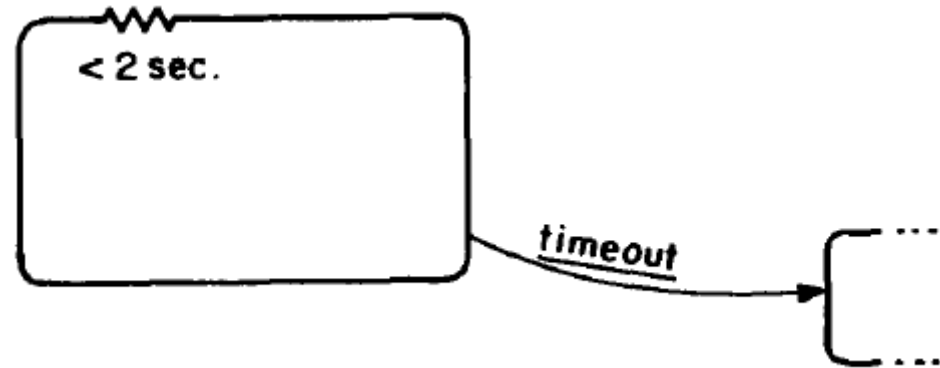
Unclustering

Il contenuto di un
superstato può
essere dettagliato al
di fuori dello
statechart in cui il
superstato compare.



Aspetti temporali: delay e timeout

- E' possibile specificare che il sistema può permanere in uno stato un tempo massimo, dopo di che ne esce con un evento detto timeout
- E' anche possibile specificare un tempo minimo di permanenza dello stato (ad esempio $> 2 \text{ sec.}$)



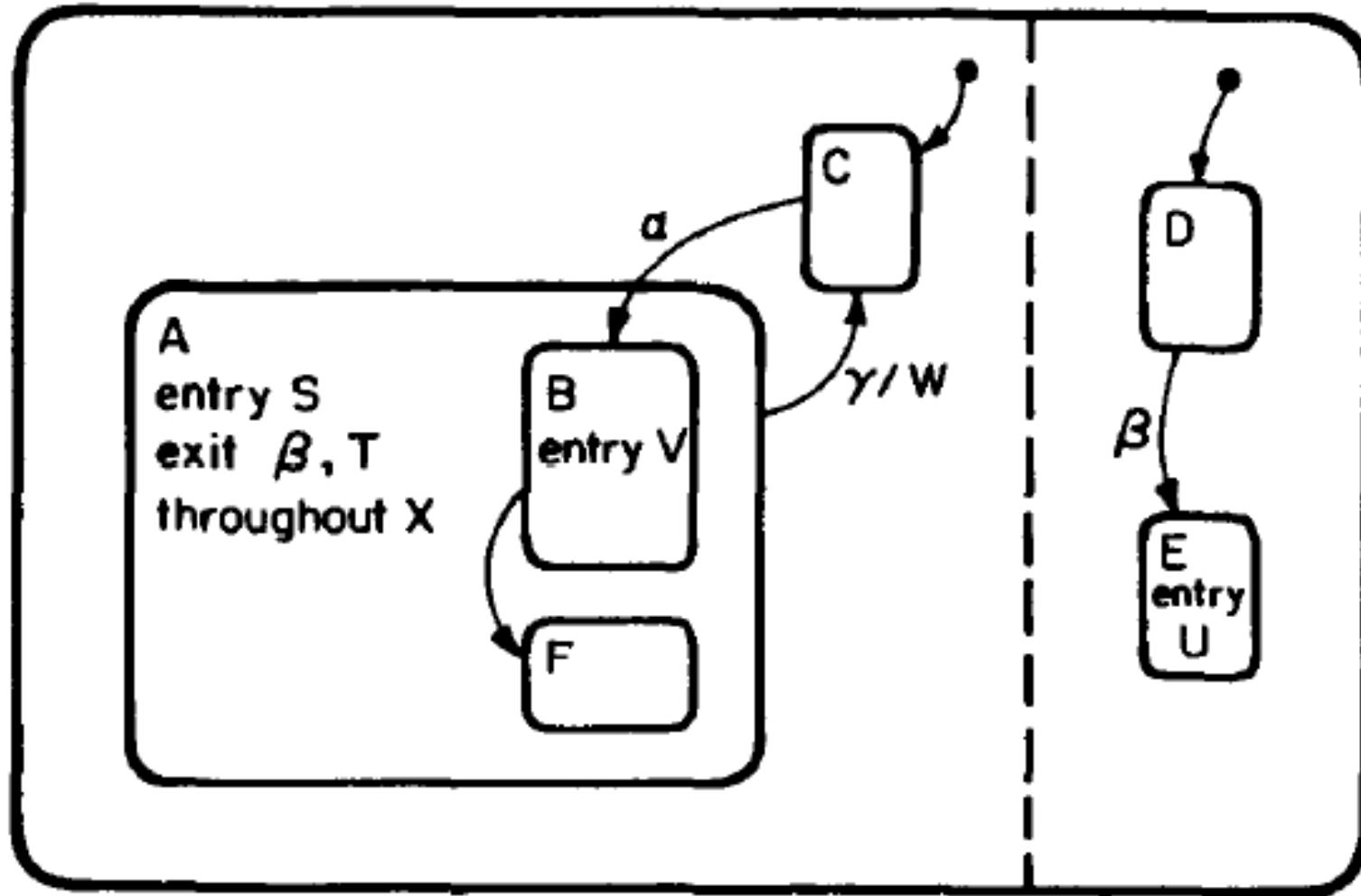
Azioni

- Finora abbiamo visto come uno statechart risponde ad eventi generati esternamente.
- E' però possibile associare alle transizioni anche delle *azioni*, cioè eventi (istantanei) generati dallo statechart.
- Le azioni, essendo eventi, possono causare transizioni di stato in altre parti dello statechart.
- Graficamente, un'azione S associata a una transizione T si indica marcando l'arco corrispondente con T/S.
- Uno stato si può annotare con la clausola entry(S) o exit(S) per indicare che l'azione S viene eseguita all'ingresso nello stato o all'uscita da esso.

Attività

- A differenza delle azioni, le attività non sono istantanee, ma hanno un inizio e una fine.
- A una attività X sono associate
 - le azioni $start(X)$ e $stop(X)$ che la iniziano e la terminano
 - la condizione $active(X)$ che indica se l'attività è in corso
- Associando a uno stato la clausola $throughout(X)$, si indica che l'attività X è dura per tutta la permanenza nello stato, cioè che le azioni $start(X)$ e $stop(X)$ vengono eseguite rispettivamente all'ingresso e all'uscita.

Azioni e attività: esempio



Valutazione

- Gli stessi vantaggi delle FSM: specifica
 - formale (eccetto per gli aspetti temporali, comunque delegabili all'implementazione)
 - grafica
 - completa
- In più
 - scalano meglio con la complessità del sistema
 - riflettono anche visivamente la composizione del sistema
 - hanno elementi che li semplificano (history)
- Richiedono specifica completa (non bottom-up, cioè non adatti a modellare codice esistente)

Implementazione

- Leggermente più complessa rispetto a quella delle FSM
- XState, libreria Javascript e Typescript
- <https://xstate.js.org/>
- Adatta soprattutto per implementare interfacce grafiche in combinazione con librerie di visualizzazione come
 - React
 - Angular
 - Vue